Librería Stratio Connector

- Introducción
- Características
- Prerequisitos
- Dependencias
- Configuración
- Implementación en Microservicios

Introducción

La librería **mdw-dm-lib-stratio-connector** es un artefacto utilitario que permite a los desarrolladores ahorrar tiempo y recursos en el proceso de conexión a la data Rest de RDM, los cuales permite consultar los catálogos en Stratio.

Características

- Librería agnóstica a Spring boot 3.4.2.
- Compilada en versión Java 21.
- Usa el paradigma de programación Reactiva (Spring Webflux).
- Maneja tres niveles de logs, info, error y warn, los logs se imprimirán en la consola de los pods.

Prerequisitos

Antes de agregar la dependencia de la librería **mdw-dm-lib-stratio-connector** al proyecto, se debe asegurar que el proyecto cuente con los siguientes requisitos:

- Plugin Spring boot versión 3.4.2 o posterior.
- Compilado en versión Java 21.
- Use el paradigma de programación Reactiva (Spring Webflux).

Dependencias

Copiar la siguiente dependencia según el tipo de compilación:

Gradle

Maven

```
<dependency>
```

```
implementation 'com.pichincha.dm.lib:mdw-dm
```

```
  <groupId>com.pichincha.dm.lib</groupId>
  <artifactId>mdw-dm-lib-stratio-connector<
  <version>1.0.0</version>
</dependency>
```

Configuración

1. Configuración del archivo application.yml

```
spring:
  data:
    #Configuración de Reactive Redis (Lettuce)
    redis:
      enabled: true
      ttl-data-duration: 4h
      ttl-oauth2-duration-to-subtract: 5m
      ssl:
        enabled: false
      database: 0
      timeout: 100ms
      connect-timeout: 5s
      host: localhost
      port: 30007
      username: someUser
      password: somePa$$word
      lettuce:
        shutdown-timeout: 200ms
        verify-peer: false
        client-options:
          disconnected-behavior: REJECT_COMMANDS
          suspend-reconnect-on-protocol-failure: false
          socket-options:
            keep-alive: false
            tcp-no-delay: false
          timeout-options:
            fixed-timeout: 5s
  #Configuración de Spring Security OAuth2 Client
  security:
    oauth2:
      client:
        registration:
          oauth2-pichincha-client:
            client-id: some-client.id
            client-secret: some-client-secret
            client-name: some-client-name
        provider:
          oauth2-pichincha-provider:
            token-uri: http://localhost:8081/cas-operacional/oauth2.0/token
      clock-skew: 1m
      scheduling:
        clock-skew: 4h

#Configuración del WebClient para el consumo de los servicios de RDM
webclient:
  connection-timeout: 200
  read-timeout: 200ms
  max-connections: -1
  max-idle-time:
  pending-acquire-max-count: -1
  pending-acquire-timeout:
  stratio:
    base-url:

#Configuración de ejecucíon del cron para validar y actualizar el access token en cache.
scheduling:
```

```yaml
  access-token:
    fixed-rate: 7200
    initial-delay: 10

#Configuración del nombre del header para obtención del Trace ID.
web-filter:
  trace-id-header-name: x-guid

#Configuración del parámetros del paginador
stratio-pager:
  offset-param-name: offset
  limit-param-name: limit

#Configuración de Circuit Breaker
resilience4j:
  circuitbreaker:
    instances:
      stratio:
        failure-rate-threshold: 1
        wait-duration-in-open-state: 1s
        permitted-number-of-calls-in-half-open-state: 5
        sliding-window-size: 10
        slow-call-rate-threshold: 1
        slow-call-duration-threshold: 300ms
        record-exceptions:
          - java.io.IOException
          - io.netty.handler.timeout.ReadTimeoutException
          - org.springframework.web.reactive.function.client.WebClientResponseException
          - org.springframework.web.reactive.function.client.WebClientRequestException

```

2. Atributos:

2.1 Reactive Redis:

> *Si el campo* `spring.data.redis.enabled` se establece en `false`, *no será necesario agregar el resto de configuraciones de* `spring.data.redis`

Campo	Requerido	Tipo	Default	Descripción
`spring.data.redis.enabled`	**NO**	***Boolean***	`false`	Indica si se debe habilitar la configuración de Reactive Redis. Si está establecido en `true`, se guardará en cache el access token obtenido del cas operacional; de lo contratio, no se habilitará la

				configuracíon de Reactive Redis.
`spring.data.redis.ttl-data-duration`	**NO**	***Duration***	`4h`	Tiempo de vida **(TTL)** de la data obtenida de RDM almacenada en Reactive Redis.
`spring.data.redis.ttl-oauth2-duration-to-subtract`	**NO**	***Duration***	`5m`	Este parámetro se utiliza para substraer el tiempo de vida **(TTL)** del access token almacenado en Reactive Redis.
`spring.data.redis.ssl.enabled`	**NO**	***Boolean***	**-**	Indica si se debe habilitar el uso de SSL para conexiones seguras a Redis. Si está establecido en `true`, se utilizará una conexión cifrada; de lo contrario, se utilizará una conexión sin cifrar.
`spring.data.redis.database`	**NO**	***Integer***	**-**	Especifica el número de la base de datos de Redis a la que se conectará la aplicación. Redis permite múltiples bases de datos, y el índice comienza en 0.
`spring.data.redis.timeout`	**NO**	***Duration***	`100ms`	Define el tiempo máximo que se esperará por una respuesta de Redis antes de considerar que ha ocurrido un tiempo de espera.

`spring.data.redis.connect-timeout`	**NO**	***Duration***	**-**	Establece el tiempo máximo para intentar conectarse a Redis. Si no se puede establecer una conexión en este tiempo, se lanzará una excepción.
`spring.data.redis.host`	**SI**	***String***	**-**	Especifica la dirección del host donde se está ejecutando el servidor Redis.
`spring.data.redis.port`	**SI**	***Integer***	**-**	Define el puerto en el que el servidor Redis está escuchando.
`spring.data.redis.username`	**NO**	***String***	**-**	Proporciona el nombre de usuario que se utilizará para autenticar la conexión con el servidor Redis. Esto es relevante si Redis se ha configurado para requerir autenticación de usuario.
`spring.data.redis.password`	**NO**	***String***	**-**	Especifica la contraseña necesaria para autenticar la conexión con el servidor Redis. Asegúrate de manejar las contraseñas de manera segura.

`spring.data.redis.lettuce.shutdown-timeout`	**NO**	***Duration***	`200ms`	Define el tiempo de espera para cerrar la conexión de Lettuce antes de forzar el cierre. Esto permite que las tareas en curso se completen adecuadamente.
`spring.data.redis.lettuce.verify-peer`	**NO**	***Boolean***	**-**	Indica si se debe verificar la identidad del servidor Redis cuando se utiliza SSL. Si se establece en `true`, se comprobará el certificado del servidor.
`spring.data.redis.lettuce.client-options.disconnected-behavior`	**NO**	***DisconnectedBehavior***	`REJECT_COMMANDS`	Configura el comportamiento del cliente cuando está desconectado. `REJECT_COMMANDS` indica que se rechazarán los comandos hasta que se restablezca la conexión.
`spring.data.redis.lettuce.client-options.suspend-reconnect-on-protocol-failure`	**NO**	***Boolean***	`false`	Indica si se debe suspender el intento de reconexión si hay un fallo en el protocolo. Si se establece en `false`, el cliente intentará reconectarse incluso tras un fallo de protocolo.

`spring.data.redis.lettuce.client-options.socket-options.keep-alive`	**NO**	***Boolean***	**-**	Especifica si se debe habilitar el keep-alive de las conexiones TCP. Esto puede ayudar a mantener la conexión abierta en redes inactivas.
`spring.data.redis.lettuce.client-options.socket-options.tcp-no-delay`	**NO**	***Boolean***	**-**	Si se establece en `true`, habilita el algoritmo de Nagle, que puede mejorar el rendimiento en conexiones de baja latencia al agrupar pequeños paquetes de datos.
`spring.data.redis.lettuce.client-options.timeout-options.fixed-timeout`	**NO**	***Duration***	**-**	Configura un tiempo de espera fijo para las operaciones de Redis. Si una operación no se completa en este tiempo, se lanzará una excepción.

2.2 Spring Security OAuth2 **(Cas Operacional)**:

Campo	Requerido	Tipo	Default	Descripción

`spring.security.oauth2.client.registration.oauth2-pichincha-client.client-id`	**SI**	***String***	**-**	Este es el identificador único asignado al cliente por el proveedor de OAuth2. Debe ser proporcionado por el proveedor y se utiliza para identificar la aplicación durante el proceso de autenticación.
`spring.security.oauth2.client.registration.oauth2-pichincha-client.client-secret`	**SI**	***String***	**-**	Es la clave secreta asociada al cliente. Similar al `client-id`, esta información es proporcionada por el proveedor de OAuth2 y se utiliza para autenticar la solicitud del cliente.
`spring.security.oauth2.client.registration.oauth2-pichincha-client.client-name`	**SI**	***String***	**-**	Proporciona un nombre legible para el cliente. Este nombre es utilizado para generar el cache key para almacenar el access token. Este nombre debe ser unico por cada microservicio.

`spring.security.oauth2.client.provider.oauth2-pichincha-provider.token-uri`	**SI**	***String***	**-**	Especifica la URI del endpoint del proveedor donde se pueden obtener los tokens de acceso. Este endpoint es utilizado por el cliente para intercambiar el código de autorización por un token de acceso durante el flujo de OAuth2 **(Cas Operacional)**.
`spring.security.oauth2.clock-skew`	**NO**	***Duration***	`1m`	Este parámetro se utiliza para definir un margen de tiempo que se considera al verificar si un token ha expirado.
`spring.security.oauth2.scheduling.clock-skew`	**NO**	***Duration***	`4h`	Este parámetro se utiliza para definir un margen de tiempo que se considera al verificar si un token ha expirado.

2.3 WebClient **(Servicios MDM)**:

Campo	Requerido	Tipo	Default	Descripción
`webclient.connection-timeout`	**NO**	***Integer***	`200`	Especifica el tiempo máximo que el cliente esperará para establecer una conexión con el servidor. Si la conexión no se establece dentro de este tiempo, se

				lanzará una excepción de tiempo de espera. *El tiempo esta definido en **milisegundos**.*
`webclient.read-timeout`	**NO**	***Duration***	`200ms`	Define el tiempo máximo que el cliente esperará para recibir una respuesta del servidor después de haber establecido la conexión. Si el servidor no responde dentro de este período, se lanzará una excepción de tiempo de espera.
`webclient.max-connections`	**NO**	***Integer***	`-1`	Define el número máximo de conexiones simultáneas que el cliente puede abrir hacia un servidor específico.
`webclient.max-idle-time`	**NO**	***Duration***	**-**	Especifica el tiempo máximo que una conexión puede estar inactiva (sin ser utilizada) antes de ser cerrada automáticamente.
`webclient.pending-acquire-max-count`	**NO**	***Integer***	`-1`	Define el número máximo de solicitudes que pueden estar en espera para adquirir una conexión del pool de conexiones.

`webclient.pending-acquire-timeout`	**NO**	***Duration***	**-**	Especifica el tiempo máximo que una solicitud puede esperar para adquirir una conexión del pool. Si se alcanza este tiempo, se lanzará una excepción.
`webclient.stratio.base-url`	**SI**	***String***	**-**	Define la URL base que se utilizará como prefijo para todas las solicitudes realizadas a través del `WebClient`. *Debe ser la URL base para el consumo de los* ***servicios de RDM.***

2.4 Circuit Breaker:

Campo	Requerido	Tipo	Default	Descripción
`resilience4j.circuitbreaker.instances.stratio.failure-rate-threshold`	**NO**	***Float***	`1`	Define el umbral de tasa de fallos que, si se supera, provoca que el Circuit Breaker cambie a un estado **"OPEN"**. Se expresa como un porcentaje y determina la proporción de llamadas fallidas respecto al total de llamadas realizadas.
`resilience4j.circuitbreaker.instances.stratio.`	**NO**	***Duration***	`1s`	Especifica el tiempo que el Circuit Breaker permanecerá en estado **"OPEN"** antes

`wait-duration-in-open-state`				de permitir una nueva verificación de las llamadas al servicio.
`resilience4j.circuitbreaker.instances.stratio.permitted-number-of-calls-in-half-open-state`	**NO**	***Integer***	`5`	Define el número máximo de llamadas permitidas al servicio cuando el Circuit Breaker se encuentra en estado **"HALF_OPEN"**.
`resilience4j.circuitbreaker.instances.stratio.sliding-window-size`	**NO**	***Integer***	`10`	Este parámetro establece el tamaño de la ventana deslizante utilizada para calcular las métricas de las llamadas al servicio (éxitos y fallos).
`resilience4j.circuitbreaker.instances.stratio.slow-call-rate-threshold`	**NO**	***Float***	`1`	Define el umbral de tasa de llamadas lentas que, si se supera, activa el Circuit Breaker para tratar estas llamadas como fallidas.
`resilience4j.circuitbreaker.instances.stratio.slow-call-duration-threshold`	**NO**	***Duration***	`300ms`	Especifica la duración máxima permitida para las llamadas antes de que se consideren "lentas". Si una llamada excede este tiempo, se cuenta como una llamada lenta.
`resilience4j.circuitbreaker`	**NO**	***Throwable[]***	`- j` `- i` `- o`	Este parámetro permite especificar una lista de

`.instances.stratio.record-exceptions`			`4 - o`	excepciones que se deben registrar como fallos en el Circuit Breaker.

2.5 Configuración CRON (Scheduling)

Campo	Requerido	Tipo	Default	Descripción
`scheduling.access-token.fixed-rate`	**NO**	***Long***	`7200`	Define el intervalo de tiempo en segundos entre el inicio de ejecuciones consecutivas de la tarea programada.
`scheduling.access-token.initial-delay`	**NO**	***Long***	`10`	Especifica el tiempo en segundos que debe esperar antes de ejecutar la tarea por primera vez.

2.6 Configuración Paginación

Campo	Requerido	Tipo	Default	Descripción
`stratio-pager.offset-param-name`	**NO**	***String***	`offset`	Define el nombre del query param asociado parámetro offset.
`stratio-pager.limit-param-name`	**NO**	***String***	`limit`	Define el nombre del query param asociado parámetro limit.

2.7 Configuración común:

Campo	Requerido	Tipo	Default	Descripción
`web-filter.trace-id-header-name`	**NO**	***String***	`x-guid`	Especifica el nombre del header para la obtención del **Trace ID**, el cual nos ayudara a identificar la trazabilidad a nivel logs.

Implementación en Microservicios

1. Detalle de los componentes de la librería:

Nombre	Descripción
`StratioQueryExecutor`	La interfaz StratioQueryExecutor es utilizada para definir métodos que permiten la obtención de datos desde RDM o Redis, aplicando configuraciones y filtros específicos, y manejando posibles excepciones.
`retrieveMono`	Método de la interfaz StratioQueryExecutor que permite obtener un dato único de los catálogos de RDM o de REDIS. **Parámetros:** • **StratioQuery stratioQuery*:*** Record que contiene el path, queryParams y headers para la obtención de datos de RDM. • **Class<T> responseType*:*** Tipo de clase que espera deserializar el componente.

	• **Retorno:** Devuelve un Mono<T>, que es un objeto único obtenido del servicio RDM.
	• **Excepciones:**
	  ◦ Puede lanzar un **ResponseStatusException** con error **503 SERVICE_UNAVAILABLE** en caso que el estatus de Circuit Breaker se encuentre **"OPEN"**.
	  ◦ Puede lanzar un **OAuth2AuthorizationExceptio n** en caso que el Cas operacional regrese un access token invalido.

`retrieveFlux`	Método de la interfaz StratioQueryExecutor que permite obtener los datos de catálogos de RDM o de REDIS. **Parámetros:** • **StratioQuery stratioQuery*:*** Record que contiene el path, queryParams y headers para la obtención de datos de RDM. • **Class<T> responseType*:*** Tipo de clase que espera deserializar el componente. • **Retorno:** Devuelve un Flux<T>, que es una lista de objetos obtenidos del servicio RDM. • **Excepciones:** ◦ Puede lanzar un **ResponseStatusException** con error **503 SERVICE_UNAVAILABLE** en

	caso que el estatus de Circuit Breaker se encuentre **“OPEN“**. ◦ Puede lanzar un **OAuth2AuthorizationException** en caso que el Cas operacional regrese un access token invalido.

2. Implementación
   a. Inyectar la dependencia `StratioQueryExecutor` en el respectivo adaptador, e invocar el método `retrieveMono` o `retrieveFlux` (Dependiendo el escenario).

> *La interfaz* `StratioQueryExecutor` *recibe una clase de tipo genérica que representa el ResponseBody de RDM, para este ejemplo se utiliza* `AccountDetail`

Ejemplo consumo endpoint **GET** en RDM:

```
package com.pichincha.dm.cura.rt.customer.position.infrastructure.output.adapter;

import com.pichincha.dm.cura.rt.customer.position.application.output.port.AccountDetailPort;
import com.pichincha.dm.cura.rt.customer.position.domain.AccountDetail;
import com.pichincha.dm.lib.stratio.connector.application.output.port.StratioQueryExecutor;
import org.springframework.stereotype.Component;

@Component
public class AccountDetailAdapter implements AccountDetailPort {

    private final StratioQueryExecutor<AccountDetail> stratioQueryExecutor;

    public AccountDetailAdapter(StratioQueryExecutor<AccountDetail> stratioQueryExecutor){
        this.stratioQueryExecutor = stratioQueryExecutor;
    }

    @Override
    public Flux<AccountDetail> retrieveAccountDetail(String partyIdentificationIdentifierValu
        var queryParams = new LinkedMultiValueMap<String, String>();
        return stratioQueryExecutor.retrieveFlux(StratioQuery
                        .newGetMethodInstance("/some/path", queryParams), AccountDetail.class
    }
}
```

Ejemplo consumo endpoint **POST** en RDM:

```
package com.pichincha.dm.cura.rt.customer.position.infrastructure.output.adapter;

import com.pichincha.dm.cura.rt.customer.position.application.output.port.AccountDetailPort;
import com.pichincha.dm.cura.rt.customer.position.domain.AccountDetail;
import com.pichincha.dm.lib.stratio.connector.application.output.port.StratioQueryExecutor;
import org.springframework.stereotype.Component;

@Component
public class AccountDetailAdapter implements AccountDetailPort {

    private final StratioQueryExecutor<AccountDetail> stratioQueryExecutor;
```

```

    public AccountDetailAdapter(StratioQueryExecutor<AccountDetail> stratioQueryExecutor){
        this.stratioQueryExecutor = stratioQueryExecutor;
    }

    @Override
    public Flux<AccountDetail> retrieveAccountDetail(String partyIdentificationIdentifierValu
        return stratioQueryExecutor.retrieveFlux(StratioQuery
                        .newPostMethodInstance("/some/path", new SomeObject("John Doe")), Acc
    }
}
```